JavaScript Practice Notes

Prime Numbers • Even / Odd • Tables • Fibonacci • manifest.json

1. Prime Number in JavaScript

Check if a number is prime

```
function isPrime(n) {
  if (n < 2) return false;
  if (n === 2) return true;
  if (n % 2 === 0) return false;

  for (let i = 3; i * i <= n; i += 2) {
    if (n % i === 0) return false;
  }
  return true;
}

console.log(isPrime(7));   // true
console.log(isPrime(10));  // false
console.log(isPrime(97));  // true
```

How it works

- Numbers below 2 are not prime.
- 2 is the only even prime, so every other even number is rejected right away.
- For odd numbers, you only need to test divisors up to the square root of n (`i * i <= n`), and you can skip even divisors by stepping `i += 2`. This is much faster than looping up to n.

All primes up to a limit (Sieve of Eratosthenes)

```
function primesUpTo(limit) {
  const isPrime = new Array(limit + 1).fill(true);
  isPrime[0] = isPrime[1] = false;

  for (let i = 2; i * i <= limit; i++) {
    if (isPrime[i]) {
      for (let j = i * i; j <= limit; j += i) {
        isPrime[j] = false;
      }
    }
  }

  return isPrime
    .map((p, i) => (p ? i : null))
    .filter(n => n !== null);
}

console.log(primesUpTo(50));
// [2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47]
```

Use the first function to test a single number, and the sieve when you need many primes at once.

2. Even or Odd Number

Using the remainder operator

```
function checkEvenOdd(n) {
  return n % 2 === 0 ? "even" : "odd";
}

console.log(checkEvenOdd(4));   // even
console.log(checkEvenOdd(7));   // odd
console.log(checkEvenOdd(0));   // even
console.log(checkEvenOdd(-3));  // odd
```

How it works

- If a number divided by 2 leaves a remainder of 0, it is even. Otherwise it is odd.
- For negative numbers, `-3 % 2` gives `-1` (not 1), which is why the check uses `=== 0` for even instead of testing `=== 1` for odd.

Shorter alternatives

```
const isEven = n => n % 2 === 0;
const isOdd = n => Math.abs(n % 2) === 1;

// Bitwise version (works for integers)
const isEvenBit = n => (n & 1) === 0;
```

Taking input from the user (browser)

```
const num = Number(prompt("Enter a number:"));

if (Number.isInteger(num)) {
  alert(`${num} is ${num % 2 === 0 ? "even" : "odd"}`);
} else {
  alert("Please enter a whole number.");
}
```

Even/odd only makes sense for whole numbers, so the `Number.isInteger` check rejects things like 3.5 or non-numeric input.

3. Calculating Multiplication Tables

Basic table with a loop

```
function printTable(n, upTo = 10) {
  for (let i = 1; i <= upTo; i++) {
    console.log(`${n} x ${i} = ${n * i}`);
  }
}

printTable(5);
// 5 x 1 = 5
// 5 x 2 = 10
// ...
// 5 x 10 = 50
```

Taking input from the user (browser)

```
const n = Number(prompt("Enter a number:"));

if (Number.isNaN(n)) {
  alert("Please enter a valid number.");
} else {
  let result = "";
  for (let i = 1; i <= 10; i++) {
    result += `${n} x ${i} = ${n * i}\n`;
  }
  alert(result);
}
```

Compact version using map

```
const table = (n, upTo = 10) =>
  Array.from({ length: upTo }, (_, i) => `${n} x ${i + 1} = ${n * (i + 1)}`);

console.log(table(7).join("\n"));
```

Full grid, tables 1 to 10 together

```
for (let i = 1; i <= 10; i++) {
  let row = "";
  for (let j = 1; j <= 10; j++) {
    row += String(i * j).padStart(4);
  }
  console.log(row);
}
```

The `padStart(4)` call keeps the columns lined up neatly.

4. Fibonacci Series

Series by number of terms

```
function fibonacci(count) {
  const series = [];
  let a = 0, b = 1;

  for (let i = 0; i < count; i++) {
    series.push(a);
    [a, b] = [b, a + b];
  }
  return series;
}

console.log(fibonacci(10));
// [0, 1, 1, 2, 3, 5, 8, 13, 21, 34]
```

How it works

- The series starts with 0 and 1, and each next number is the sum of the previous two.
- `[a, b] = [b, a + b]` updates both values at once, so you do not need a temporary variable.

Series up to a limit (instead of a count)

```
function fibonacciUpTo(limit) {
  const series = [];
  let a = 0, b = 1;

  while (a <= limit) {
    series.push(a);
    [a, b] = [b, a + b];
  }
  return series;
}

console.log(fibonacciUpTo(100));
// [0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89]
```

Recursive version (nth number)

```
function fib(n) {
  if (n <= 1) return n;
  return fib(n - 1) + fib(n - 2);
}

console.log(fib(10)); // 55
```

The recursive version is simple but slow for large n because it recalculates the same values repeatedly. A memoized version fixes that:

```
function fibMemo(n, memo = {}) {
  if (n <= 1) return n;
  if (memo[n]) return memo[n];
  memo[n] = fibMemo(n - 1, memo) + fibMemo(n - 2, memo);
  return memo[n];
}

console.log(fibMemo(50)); // 12586269025
```

Taking input from the user (browser)

```
const count = Number(prompt("How many terms?"));

if (Number.isInteger(count) && count > 0) {
  alert(fibonacci(count).join(", "));
} else {
  alert("Please enter a positive whole number.");
}
```

5. manifest.json

The `manifest.json` file is used in two common places: web apps (PWA) and Chrome extensions.

Web app manifest (PWA)

This file tells the browser how your website should behave when installed on a phone or desktop, like an app.

```
{
  "name": "My Calculator App",
  "short_name": "Calc",
  "description": "Prime, even/odd, tables and Fibonacci tools",
  "start_url": "/index.html",
  "display": "standalone",
  "background_color": "#ffffff",
  "theme_color": "#0d6efd",
  "icons": [
    {
      "src": "icon-192.png",
      "sizes": "192x192",
      "type": "image/png"
    },
    {
      "src": "icon-512.png",
      "sizes": "512x512",
      "type": "image/png"
    }
  ]
}
```

Link it in the <head> of your HTML:

```
<link rel="manifest" href="manifest.json">
<meta name="theme-color" content="#0d6efd">
```

- `name` / `short_name`: the app name shown on the home screen.
- `start_url`: the page that opens when the app is launched.
- `display`: `standalone` hides the browser bar so it feels like a native app.
- `icons`: at least 192x192 and 512x512 for installability.

To make it fully installable, you also need HTTPS (or localhost) and usually a service worker.

Chrome extension manifest

```
{
  "manifest_version": 3,
  "name": "My Extension",
  "version": "1.0",
  "description": "A simple extension",
  "action": {
    "default_popup": "popup.html"
  },
  "permissions": ["storage"]
}
```